“Pharma Search – An Intelligent Medicine Search and Recommendation System”

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

by

MONISH PUTTI : 2520030395

HEMASAI : 2520030599

NIKHIL REDDY : 2520030560

Team Number: 18

Section: S6

Under the Guidance of

Dr.V.Sireesha

Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

2026-27

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Declaration

The Project Report entitled **"Pharma Search – An Intelligent Medicine Search and Recommendation System"** is a record of Bonafide work of **Monish Putti – 2520030395, Hemasai – 2520030599, Nikhil Reddy – 2520030560** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering. The results embodied in this report have not been copied from any other departments/University/Institute.

Monish Putti – 2520030395
Hemasai – 2520030599
Nikhil Reddy – 2520030560

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is to certify that the miniproject based report entitled **"Pharma Search – An Intelligent Medicine Search and Recommendation System"** is a bonafide work done and submitted by **MONISH PUTTI – 2520030395, HEMASAI – 2520030599, NIKHIL REDDY – 2520030560** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, during the academic year **2026-27** .

Signature of the Guide
(Dr. V.Sireesha)

Signature of the HOD

Signature of the External Examiner

ACKNOWLEDGEMENT

We express our sincere gratitude to our guide, **Dr. S. Vinay Kumar, Associate Professor, Department of Computer Science and Engineering**, for his valuable guidance, continuous support, feedback, and encouragement throughout the development of Pharma Search. His suggestions helped us improve the system design, the algorithmic implementation, and the documentation of this project.

We also express our sincere thanks to **Dr. P. Venkateshwara Rao, Associate Professor and HOD, Department of Computer Science and Engineering**, for providing the academic environment and support required to complete this project.

We are grateful to the department and the university for providing the resources and opportunities necessary for successfully carrying out this work, and to our teammates for their cooperation throughout the project.

Monish Putti – 2520030395
Hemasai – 2520030599
Nikhil Reddy – 2520030560

Abstract

Pharma Search – An Intelligent Medicine Search and Recommendation System is a web-based medicine information search application designed to provide fast and relevant results from a large medicine corpus. The system processes **242,381 medicine records** and supports search across medicine name, composition, manufacturer, type, and pack size.

The search engine combines **Knuth-Morris-Pratt (KMP), Rabin-Karp, Levenshtein Edit Distance, and Cosine Similarity** . KMP provides the main exact-pattern score, Rabin-Karp provides additional rolling-hash pattern detection, Edit Distance supports spelling-tolerant matching, and Cosine Similarity measures textual similarity across medicine fields. The final ranking uses **40% KMP, 30% fuzzy matching, and 30% cosine similarity** .

The project is implemented using **Java 17, Spring Boot, HTML, CSS, JavaScript, and file-based CSV/TXT storage**. The frontend provides search, filters, sorting, favourites, and medicine details, while the backend exposes REST APIs. Additional DSA-3 implementations demonstrate the **Z-Algorithm, optimized dynamic programming, Edmonds-Karp maximum flow, greedy approximation, randomized sampling, and parallel prefix sum** concepts.

Table of Contents

			Page No.
Acknowledgement			
Abstract			
Table of Contents			
List of Figures			
1	Introduction	1.1 Overview 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives	1–3
2	System Requirements	2.1 Functional Requirements 2.2 Non-Functional Requirements 2.3 Software and Hardware Requirements 2.4 Dataset Requirements 2.5 Existing Work and Research Gaps	4–6
3	System Design	3.1 High-Level Architecture 3.2 Description of Each Layer 3.3 Data Flow 3.4 Module Mapping 3.5 Algorithm-to-Feature Mapping 3.6 Choice of Tools and Techniques	7–9
4	Implementation	4.1 Module-Wise Implementation 4.2 Front-End Logic (UI Rendering) 4.3 API Integration 4.4 Data Processing 4.5 DSA-3 CO Implementations	10–19

5	Results	5.1 Test Environment 5.2 Functional Results 5.3 Performance Observations 5.4 Screenshots and Analysis	20–28
	References	Books, papers, APIs, documentation sites used	32

List of Figures

Fig. No.	Caption	Page No.
3.1	High-Level Architecture of Pharma Search	7
5.1	Pharma Search Homepage	22
5.2	Medicine Search Result Page	22
5.3	Search Result for an Exact Medicine Name	23
5.4	Search Result for a Misspelled Medicine Query	23
5.5	Medicine Type Filtering	24
5.6	Result Sorting	24
5.7	Favourite Medicines	25
5.8	Backend Terminal Showing Medicine Dataset Loading	25
5.9	API Search Response (/api/search)	26
5.10	Z-Algorithm and String-Matching Demonstration Output	26
5.11	Edmonds-Karp Maximum Flow Demonstration Output	27
5.12	Greedy Approximation (Budget Recommendation) Output	27
5.13	Randomized Sampling Demonstration Output	28
5.14	Parallel Prefix Sum Demonstration Output	28

1. INTRODUCTION

1.1 Overview

Medicines are sold under thousands of brand names, with different compositions, manufacturers, pack sizes, and prices. Finding the right medicine record quickly in a very large catalogue is a practical information-retrieval problem that is relevant to pharmacies, online health platforms, and patients. Efficient string-processing algorithms are central to solving it [4].

Pharma Search – An Intelligent Medicine Search and Recommendation System is a Java-based web application developed to provide efficient searching and retrieval of medicine information from a large dataset. Users can search medicines by name, composition, manufacturer, type, and pack size. The project uses a cleaned medicine dataset containing **242,381 medicine records**. The backend is implemented using Java and Spring Boot [6], while HTML, CSS, and JavaScript provide a browser-based user interface.

The system applies Data Structures and Algorithms concepts to improve search accuracy and relevance. The search pipeline combines KMP string matching [3], Rabin-Karp pattern matching [7], Edit Distance [5][11], and Cosine Similarity [10] to identify and rank relevant records. An additional Z-Algorithm implementation [8] is included as part of the string-algorithm component. The system also provides medicine-type filtering, result sorting, a favourite-medicine feature, and relevance-based presentation of results.

1.2 Problem Statement

Searching for medicine information in a large dataset becomes inefficient when users enter incomplete names, spelling variations, compositions, or other medicine-related information. A simple exact text search fails when the query does not exactly match the stored record. For example, a user may enter an incorrectly spelled medicine name or search using a

composition instead of the medicine name.

The problem addressed by this project is to develop an intelligent search system that can process a large medicine dataset and retrieve relevant results using efficient string-matching and similarity algorithms. The system should:

- Search medicines using multiple attributes.
- Handle spelling variations in medicine names.
- Detect exact and approximate text matches.
- Rank search results according to relevance.
- Support medicine-type filtering and result sorting.
- Provide a simple and interactive web interface.
- Demonstrate practical applications of advanced Data Structures and Algorithms.

1.3 Scope of the Project

The scope of the project covers the development of a web-based medicine search application capable of processing a large medicine dataset. The system includes:

- CSV-based medicine dataset storage, data cleaning, and preprocessing.
- Java-based backend processing with a Spring Boot REST API and a browser-based frontend.
- Medicine search by name, composition, manufacturer, type, and pack size.
- KMP, Rabin-Karp, and Z-Algorithm string matching; Edit Distance for approximate matching; Cosine Similarity for text similarity.
- Combined relevance scoring and ranking, medicine-type filtering, result sorting, and favourite medicines.
- Additional DSA-3 implementations corresponding to CO1–CO6 (network flow, approximation, randomized and parallel algorithms).

The system is intended as an academic DSA project. It does not provide medical diagnosis or treatment advice.

Challenges involved. The main challenges are processing more than 240,000 records with acceptable response time, handling spelling variations, integrating several algorithms without breaking existing search behaviour, and keeping the frontend and backend communicating reliably. These are discussed in Chapter 6.

Organisation of the report. Chapter 2 presents the system requirements and related work. Chapter 3 describes the system design. Chapter 4 explains the implementation with pseudocode. Chapter 5 presents the results with screenshots. Chapter 6 discusses challenges and limitations, and Chapter 7 concludes the report and outlines future work.

1.4 Objectives

- To develop an intelligent medicine-search system using Java.
- To process and search a large medicine dataset efficiently.
- To implement KMP for efficient pattern matching, Rabin-Karp using rolling hashing, and the Z-Algorithm for linear-time string matching.
- To use Edit Distance for handling spelling variations and Cosine Similarity for measuring textual similarity.
- To combine multiple algorithmic scores to rank search results.
- To provide filtering and sorting functionality through a user-friendly browser-based interface.
- To demonstrate the practical application of DSA-3 concepts in a real-world search problem.

2. SYSTEM REQUIREMENTS

2.1 Functional Requirements

The functional requirements describe what the system must do. The system should:

- Load medicine records from the processed CSV dataset.

- Search medicines using a user-provided query across medicine name, compositions, manufacturer, type, and pack size.
- Perform exact pattern matching using KMP and hash-based pattern detection using Rabin-Karp.
- Calculate approximate matching using Edit Distance and textual similarity using Cosine Similarity.
- Combine algorithmic scores to rank search results.
- Return relevant medicine results through the REST API and display them through the frontend.
- Filter medicines by type such as Allopathy, Ayurveda, and Homeopathy.
- Sort results according to relevance, medicine name, and price.
- Allow users to add medicines to favourites.

2.2 Non-Functional Requirements

- **Performance:** The system should process a large medicine dataset efficiently.
- **Accuracy:** Search results should provide relevant records for exact and approximate queries.
- **Scalability:** The application should be capable of processing hundreds of thousands of medicine records.
- **Usability:** The frontend should provide a simple interface for searching and viewing medicines.
- **Maintainability:** Algorithm implementations, backend services, models, repositories, and frontend components should remain logically separated.
- **Reliability:** Invalid or incomplete records should not prevent the complete dataset from being loaded.
- **Responsiveness:** Search requests should return results without unnecessary delay.

2.3 Software and Hardware Requirements

Table 2.1 Software Requirements

Component	Specification
Programming language	Java Development Kit (JDK) 17 or newer
Framework	Spring Boot 3.5.5 (REST API on port 8080)
Build tool	Apache Maven
Frontend	HTML, CSS, and JavaScript (Fetch API)
IDE	Visual Studio Code or another Java-compatible IDE
Browser	Any modern web browser
Version control	Git and GitHub
Data storage	File-based CSV/TXT (data/processed/ medicines_clean.csv)

Table 2.2 Hardware Requirements (Minimum)

Component	Specification
Processor	Dual-core 2.0 GHz or higher
Memory (RAM)	4 GB minimum (8 GB recommended, since the dataset is held in memory)
Storage	Sufficient free disk space for the project and the medicine dataset
Display / Network	Standard display; local network access to localhost:8080

2.4 Dataset Requirements

The project uses a cleaned medicine dataset stored as data/processed/ medicines_clean.csv, containing approximately 242,381 medicine records. Each record contains the following attributes: ID, Medicine Name,

Price, Manufacturer, Type, Pack Size, Composition 1, and Composition 2. The dataset is loaded by the Java backend and converted into medicine objects for search processing.

2.5 Existing Work and Research Gaps

Conventional medicine or product search on many catalogue systems relies on exact keyword matching or substring queries (for example, SQL LIKE queries), which fail on misspelt or partial input. Full-text search engines built on inverted indexes improve retrieval and add fuzzy matching, and the information-retrieval literature uses cosine similarity over term vectors to rank documents [10]. Classical string-matching algorithms such as KMP [2][3] and Rabin-Karp [7] give guaranteed or expected linear-time pattern search, and edit distance [5][11] is the standard measure for tolerating spelling errors.

These techniques are usually studied in isolation. The gap addressed here is a single, understandable pipeline in which exact matching, hash-based detection, approximate matching, and similarity scoring are combined through a weighted relevance score over several medicine attributes, and demonstrated on a realistic dataset of more than 240,000 records. Limitations of the approach, such as predefined weights and the lack of semantic understanding, are discussed in Section 6.3.

3. SYSTEM DESIGN

3.1 High-Level Architecture

The architecture of Pharma Search is layered, as shown in Fig. 3.1. The user interacts with a web frontend, which communicates with a Spring Boot REST API. The controller passes the query to the Search Service, which uses the algorithm modules and the Medicine Repository, backed by the processed CSV dataset.

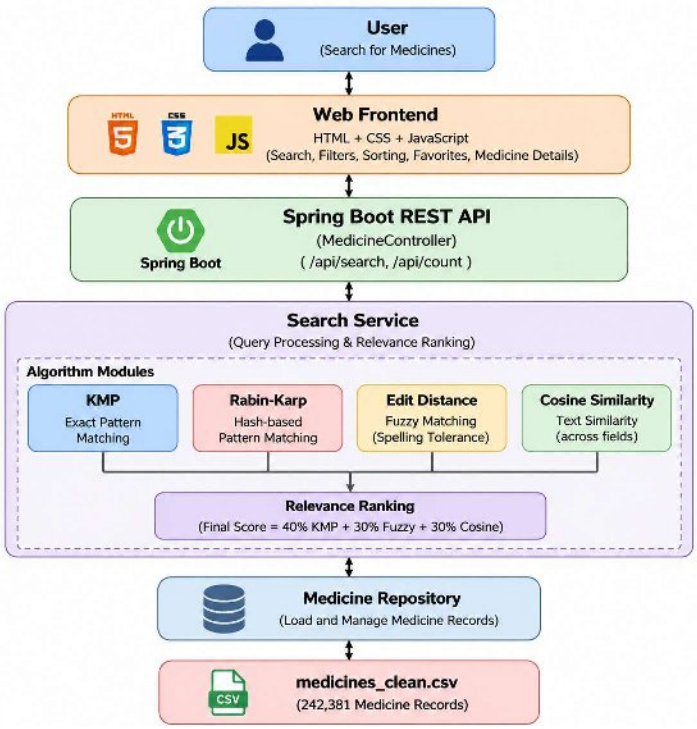


Fig. 3.1 High-Level Architecture of Pharma Search

3.2 Description of Each Layer

- **Presentation layer (Web Frontend):** HTML, CSS, and JavaScript provide the search bar, filters, sorting, favourites, and medicine details.
- **API layer (Spring Boot REST API):** MedicineController exposes /api/search and /api/count and returns JSON.
- **Service layer (Search Service):** processes the query, runs the algorithm modules, and computes the final relevance ranking.
- **Algorithm modules:** KMP (exact matching), Rabin-Karp (hash-based detection), Edit Distance (fuzzy matching), and Cosine Similarity (text similarity).
- **Data layer (Medicine Repository and CSV):** loads and manages the 242,381 medicine records held in memory.

3.3 Data Flow

The search process follows these steps:

1. The user enters a medicine-related query.

2. The frontend sends the query to /api/search.
3. The Medicine Controller receives the request.
4. The Search Service processes the query.
5. KMP checks for exact pattern matches.
6. Rabin-Karp performs additional hash-based pattern detection.
7. Edit Distance calculates approximate matching.
8. Cosine Similarity measures textual similarity.
9. The scores are combined to calculate the final relevance score.
10. Results are sorted according to relevance.
11. The top relevant medicines are returned to the frontend.
12. The frontend displays the results to the user.

3.4 Module Mapping

- **Dataset Module** – loads and processes medicine records.
- **Repository Module** – stores medicine objects in memory.
- **Search Module** – processes user queries.
- **String Algorithm Module** – implements KMP, Rabin-Karp, and Z-Algorithm.
- **Fuzzy Matching Module** – implements Edit Distance.
- **Similarity Module** – implements Cosine Similarity.
- **Ranking Module** – combines algorithmic scores.
- **Controller Module** – exposes REST API endpoints.
- **Frontend Module** – provides the browser interface.
- **Filter and Sort Module** – handles medicine type and result ordering.

3.5 Algorithm-to-Feature Mapping

Table 3.1 Algorithm-to-Feature Mapping

Algorithm	Application
KMP	Exact pattern matching

Rabin-Karp	Hash-based pattern detection
Combined Ranking	Relevance-based result ordering
Edmonds-Karp	CO4 allocation demonstration
Approximation	CO5 budget recommendation demonstration
Randomized Sampling	CO6 demonstration
Parallel Prefix Sum	CO6 parallel algorithm demonstration

For the main medicine-search pipeline, KMP, Rabin-Karp, Edit Distance, and Cosine Similarity are directly used by SearchService. The CO4–CO6 implementations provide separate demonstrations of the corresponding DSA concepts.

3.6 Choice of Tools and Techniques

Java and Spring Boot were chosen because Java suits algorithm-heavy code with strong typing, and Spring Boot provides a ready REST framework with minimal configuration [1][6]. **A CSV dataset** keeps the project simple and portable, and the whole dataset fits in memory for fast repeated searching. **KMP** gives linear-time exact matching without re-scanning text [3]; **Rabin-Karp** adds rolling-hash detection that is simple and fast on average [7]; **Edit Distance** tolerates spelling mistakes [5]; and **Cosine Similarity** scores overall textual closeness across fields [10]. Combining them with weights lets each technique cover the weakness of the others. **HTML, CSS, and JavaScript** need no extra build tooling and communicate with the backend through the Fetch API.

4. IMPLEMENTATION

This chapter explains the implementation of each module. The core algorithms are presented as pseudocode. Let n be the length of the text, m the length of the pattern, and N the number of medicine records.

4.1 Module-Wise Implementation

4.1.1 Dataset Loading and Repository

The MedicineRepository reads the CSV file, parses each record, creates a Medicine object, and stores the records in memory for searching. The CSV parser handles quoted fields, and invalid numeric records are skipped so that one malformed record does not stop the complete load.

Pseudocode 4.1: Dataset Loading

```
FUNCTION loadDataset(filePath)
    records ← empty list
    OPEN filePath for reading; SKIP header line
    FOR EACH line IN file DO
        fields ← parseCSV(line)        // handles quoted values
        IF fields is invalid OR price is not numeric THEN
            CONTINUE                   // skip malformed record
        END IF
        m ← NEW Medicine(id, name, price, manufacturer, type,
        packSize, comp1, comp2)
        ADD m TO records
    END FOR
    RETURN records
END FUNCTION
```

4.1.2 KMP Pattern Matching

KMP avoids re-comparing characters by precomputing the longest-proper-prefix-which-is-also-suffix (LPS) table of the pattern, giving $O(n + m)$ matching [2][3]. It is the main exact-match scorer in SearchService.

Pseudocode 4.2: KMP Pattern Matching

```
FUNCTION computeLPS(pattern)
    lps[0] ← 0; len ← 0; i ← 1
    WHILE i < length(pattern) DO
        IF pattern[i] = pattern[len] THEN
```

```
            len ← len + 1; lps[i] ← len; i ← i + 1
        ELSE IF len ≠ 0 THEN
            len ← lps[len - 1]
        ELSE
            lps[i] ← 0; i ← i + 1
        END IF
    END WHILE
    RETURN lps
END FUNCTION

FUNCTION KMPSearch(text, pattern)
    lps ← computeLPS(pattern); i ← 0; j ← 0; positions ← empty list
    WHILE i < length(text) DO
        IF text[i] = pattern[j] THEN i ← i + 1; j ← j + 1
        IF j = length(pattern) THEN
            ADD (i - j) TO positions; j ← lps[j - 1]
        ELSE IF i < length(text) AND text[i] ≠ pattern[j] THEN
            IF j ≠ 0 THEN j ← lps[j - 1] ELSE i ← i + 1
        END IF
    END WHILE
    RETURN positions
END FUNCTION
```

4.1.3 Rabin-Karp Pattern Matching

Rabin-Karp compares a rolling hash of each text window with the hash of the pattern, and verifies characters only when the hashes match. Its expected time is O(n + m) [7].

Pseudocode 4.3: Rabin-Karp Pattern Matching

```
FUNCTION RabinKarp(text, pattern, d, q)      // d = alphabet size, q =
prime modulus
    h ← d^(m-1) MOD q; p ← 0; t ← 0
```

```
    FOR i ← 0 TO m - 1 DO
        p ← (d·p + pattern[i]) MOD q
        t ← (d·t + text[i]) MOD q
    END FOR
    FOR s ← 0 TO n - m DO
        IF p = t THEN
            IF text[s .. s+m-1] = pattern THEN REPORT match at s
        END IF
        IF s < n - m THEN
            t ← (d·(t - text[s]·h) + text[s+m]) MOD q
            IF t < 0 THEN t ← t + q
        END IF
    END FOR
END FUNCTION
```

4.1.4 Z-Algorithm

The Z-Algorithm builds the Z-array of the string pattern + "$" + text, where Z[i] is the length of the longest substring starting ati that matches a prefix. Every index with Z[i] = m is a match, and the total time is O(n + m) [8].

Pseudocode 4.4: Z-Algorithm

```
FUNCTION buildZ(s)
    Z[0] ← length(s); l ← 0; r ← 0
    FOR i ← 1 TO length(s) - 1 DO
        IF i < r THEN Z[i] ← MIN(r - i, Z[i - l]) ELSE Z[i] ← 0
        WHILE i + Z[i] < length(s) AND s[Z[i]] = s[i + Z[i]] DO
            Z[i] ← Z[i] + 1
        END WHILE
        IF i + Z[i] > r THEN l ← i; r ← i + Z[i]
    END FOR
    RETURN Z
```

```
END FUNCTION

FUNCTION ZSearch(text, pattern)
    Z ← buildZ(pattern + "$" + text)
    FOR i ← m + 1 TO length(Z) - 1 DO
        IF Z[i] = m THEN REPORT match at (i - m - 1)
    END FOR
END FUNCTION
```

4.1.5 Edit Distance

Edit (Levenshtein) distance is the minimum number of insertions, deletions, and substitutions needed to turn one string into another. It is computed by dynamic programming in O(mn) time [5][11] and is converted into a similarity between 0 and 1 so that a misspelt query can still match the correct medicine.

Pseudocode 4.5: Edit Distance and Fuzzy Score

```
FUNCTION editDistance(a, b)
    FOR i ← 0 TO length(a) DO D[i][0] ← i
    FOR j ← 0 TO length(b) DO D[0][j] ← j
    FOR i ← 1 TO length(a) DO
        FOR j ← 1 TO length(b) DO
            cost ← 0 IF a[i-1] = b[j-1] ELSE 1
            D[i][j] ← MIN(D[i-1][j] + 1, D[i][j-1] + 1, D[i-1][j-1] + cost)
        END FOR
    END FOR
    RETURN D[length(a)][length(b)]
END FUNCTION

FUNCTION fuzzyScore(query, field)
    RETURN 1 - editDistance(query, field) / MAX(length(query),
    length(field))
```

```
END FUNCTION
```

4.1.6 Cosine Similarity

Cosine similarity treats the query and a medicine field as term-frequency vectors and measures the cosine of the angle between them [10]. A value of 1 means identical term distribution, and 0 means no common terms.

Pseudocode 4.6: Cosine Similarity

```
FUNCTION cosineSimilarity(textA, textB)
    vA ← termFrequency(tokenize(textA))
    vB ← termFrequency(tokenize(textB))
    dot ← 0
    FOR EACH term t IN vA DO
        IF t IN vB THEN dot ← dot + vA[t] · vB[t]
    END FOR
    normA ← SQRT(SUM of vA[t]² for all t)
    normB ← SQRT(SUM of vB[t]² for all t)
    IF normA = 0 OR normB = 0 THEN RETURN 0
    RETURN dot / (normA · normB)
END FUNCTION
```

4.1.7 Search Ranking

SearchService evaluates the query against the medicine name, Composition 1, Composition 2, manufacturer, pack size, and type. The final relevance score is:

Final Score = 0.40 × KMP Score + 0.30 × Edit Distance Score + 0.30 × Cosine Similarity Score

Rabin-Karp is additionally used to detect exact pattern matches.

Pseudocode 4.7: Search and Relevance Ranking

```
FUNCTION search(query, typeFilter, sortBy)
    q ← lowercase(trim(query)); results ← empty list
```

```
    FOR EACH medicine m IN repository DO
        IF typeFilter is set AND m.type ≠ typeFilter THEN CONTINUE
        best ← 0
        FOR EACH field f IN {name, comp1, comp2, manufacturer,
        packSize, type} DO
            kmp ← 1 IF KMPSearch(lowercase(f), q) is not empty ELSE 0
            IF kmp = 0 AND RabinKarp(lowercase(f), q) finds a match
            THEN kmp ← 1
            fuzzy ← fuzzyScore(q, lowercase(f))
            cos ← cosineSimilarity(q, f)
            score ← 0.40·kmp + 0.30·fuzzy + 0.30·cos
            best ← MAX(best, score)
        END FOR
        IF best > threshold THEN ADD (m, best) TO results
    END FOR
    SORT results BY sortBy       // relevance (desc) | name | price
    RETURN top K of results
END FUNCTION
```

4.2 Front-End Logic (UI Rendering)

The frontend is implemented using HTML, CSS, and JavaScript. The interface provides a medicine search bar and search button, example search queries, medicine result cards, medicine-type filters, sorting options, favourites, medicine information display, and a responsive layout. The JavaScript code communicates with the Spring Boot backend using the Fetch API and renders the returned JSON dynamically.

Pseudocode 4.8: Frontend Search and Rendering

```
ON click(searchButton) OR press(Enter):
    query ← searchInput.value
    response ← FETCH "/api/search?query=" + encode(query)
    medicines ← PARSE response AS JSON
```

```
    APPLY selected type filter AND selected sort order
    CLEAR resultContainer
    FOR EACH m IN medicines DO
        card ← CREATE result card (name, composition, manufacturer,
        type, pack, price)
        ADD favourite button to card; APPEND card TO resultContainer
    END FOR
```

4.3 API Integration

The frontend communicates with the backend through REST API endpoints: /api/search and /api/count. A search request is sent in the form /api/search?query=<search-query>. The backend processes the query and returns medicine records in JSON format, and the frontend renders the results dynamically.

4.4 Data Processing

The MedicineRepository reads the processed CSV dataset using Java file-handling classes. The CSV parser handles quoted values and separates the attributes into individual fields. Invalid numeric records are skipped during loading, and the loaded dataset is stored as Java Medicine objects for efficient repeated searching.

4.5 DSA-3 CO Implementations

In addition to the search pipeline, the project includes separate implementations that demonstrate the remaining DSA-3 course outcomes. Their pseudocode is given below.

4.5.1 Edmonds-Karp Maximum Flow (CO4)

Edmonds-Karp computes maximum flow by repeatedly finding the shortest augmenting path with breadth-first search, in $O(VE^2)$ time [9]. It is used to demonstrate allocation as a flow problem.

Pseudocode 4.9: Edmonds-Karp

```
FUNCTION edmondsKarp(G, s, t)
    FOR EACH edge (u, v) DO flow(u, v) ← 0
    maxFlow ← 0
    LOOP
        parent ← BFS(residual graph of G, s)      // shortest s→t path
        IF t is not reached THEN BREAK
        bottleneck ← MIN residual capacity along path s     t
        FOR EACH edge (u, v) ON the path DO
            flow(u, v) ← flow(u, v) + bottleneck
            flow(v, u) ← flow(v, u) - bottleneck
        END FOR
        maxFlow ← maxFlow + bottleneck
    END LOOP
    RETURN maxFlow
END FUNCTION
```

4.5.2 Greedy Approximation for Budget Recommendation (CO5)

Choosing medicines of maximum total relevance within a budget is a knapsack-type problem. The greedy approximation sorts items by relevance-to-price ratio and, by also comparing with the best single item, guarantees at least half of the optimal value [4].

Pseudocode 4.10: Greedy Approximation

```
FUNCTION budgetRecommend(items, budget)
    SORT items BY (relevance / price) DESCENDING
    total ← 0; spent ← 0; chosen ← empty list
    FOR EACH item IN items DO
        IF spent + item.price ≤ budget THEN
            ADD item TO chosen; spent ← spent + item.price; total ←
            total + item.relevance
        END IF
    END FOR
```

```
    best ← item with highest relevance AND price ≤ budget
    IF best.relevance > total THEN RETURN [best]
    RETURN chosen
END FUNCTION
```

4.5.3 Randomized Sampling (CO6)

Reservoir sampling picks k items uniformly at random from a stream or list of unknown length using one pass and O(k) memory.

Pseudocode 4.11: Randomized Sampling

```
FUNCTION sample(items, k)
    reservoir ← first k items of items
    FOR i ← k TO length(items) - 1 DO
        j ← RANDOM integer in [0, i]
        IF j < k THEN reservoir[j] ← items[i]
    END FOR
    RETURN reservoir
END FUNCTION
```

4.5.4 Parallel Prefix Sum (CO6)

The parallel prefix sum (scan) computes all running totals inO(log n) parallel steps using O(n) work per level [12].

Pseudocode 4.12: Parallel Prefix Sum

```
FUNCTION prefixSum(a)                    // a has n elements
    FOR d ← 1; d < n; d ← 2·d DO
        b ← COPY of a
        PARALLEL FOR i ← d TO n - 1 DO
            b[i] ← a[i] + a[i - d]
        END PARALLEL FOR
        a ← b
    END FOR
```

```
    RETURN a
END FUNCTION
```

5. RESULTS

5.1 Test Environment

The system was tested using the Java Spring Boot backend and a browser-based frontend. The processed medicine dataset contains approximately 242,381 records. The backend successfully loads the complete dataset and exposes the search functionality through the REST API.

5.2 Functional Results

The implemented system successfully demonstrates:

- Loading of 242,381 medicine records.
- Medicine search through the web interface.
- Exact medicine-name, approximate spelling-based, composition-based, and manufacturer-based searching.
- Medicine type filtering and result sorting.
- Favourite medicine functionality.
- API-based communication between frontend and backend.

The search system also returns relevant results for queries containing spelling variations.

Table 5.1 Functional Test Cases

No.	Test Case	Expected Behaviour	Result
1	Exact medicine name	The matching medicine appears at the top of the results	
2	Misspelt medicine name	The intended medicine is still returned (Edit Distance)	

3	Composition query	Medicines containing the composition are listed	
4	Manufacturer query	Medicines from that manufacturer are listed	
5	Type filter (e.g., Ayurveda)	Only medicines of the selected type are shown	
6	Sort by price / name	Results are reordered accordingly	
7	Add to favourites	Medicine is saved to the favourites list	
8	GET /api/count	Returns the total number of loaded records (242,381)	

5.3 Performance Observations

The system is designed to process a large medicine dataset containing hundreds of thousands of records. KMP provides linear-time pattern matching after preprocessing, while Rabin-Karp uses rolling hashing for pattern detection. Edit Distance provides approximate matching, while Cosine Similarity contributes to relevance ranking. The combination of these algorithms allows the system to provide more useful results than a simple exact string search.

Table 5.2 Time Complexity of the Algorithms Used

Algorithm	Time Complexity	Role
KMP	O(n + m)	Exact matching
Rabin-Karp	O(n + m) expected	Hash-based detection
Z-Algorithm	O(n + m)	Pattern matching
Edit Distance	O(m · n)	Fuzzy matching
Cosine Similarity	O(m + n) terms	Text similarity

Edmonds-Karp	O(V · E²)	Max flow

5.4 Screenshots and Analysis

The following figures show the working of the system. Each box below is reserved for a screenshot, to be placed inside the box and followed by a short observation.

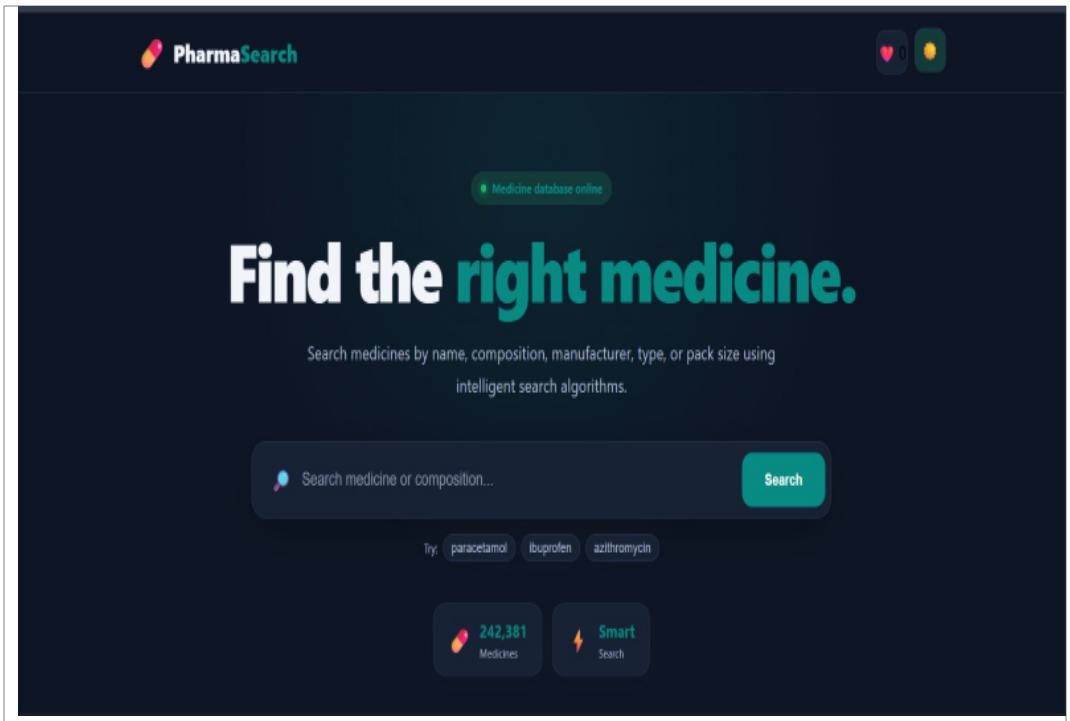


Fig. 5.1 Pharma Search Homepage

Analysis: _____

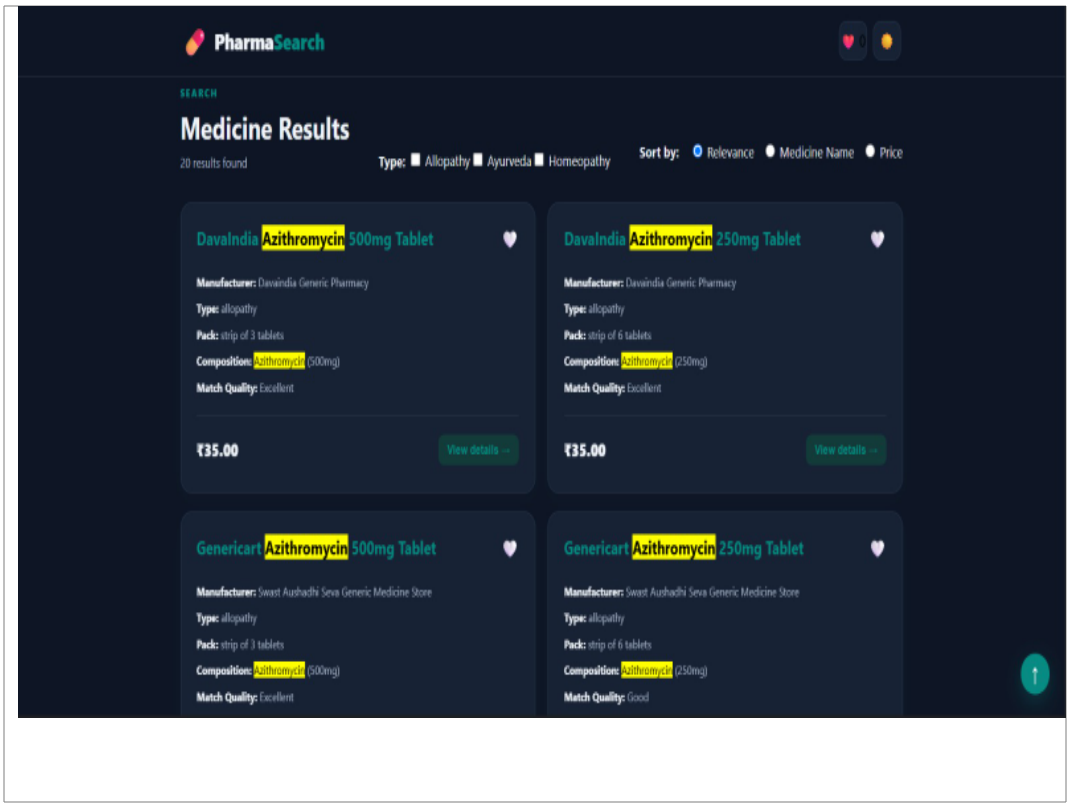


Fig. 5.2 Medicine Search Result Page

Analysis: _____

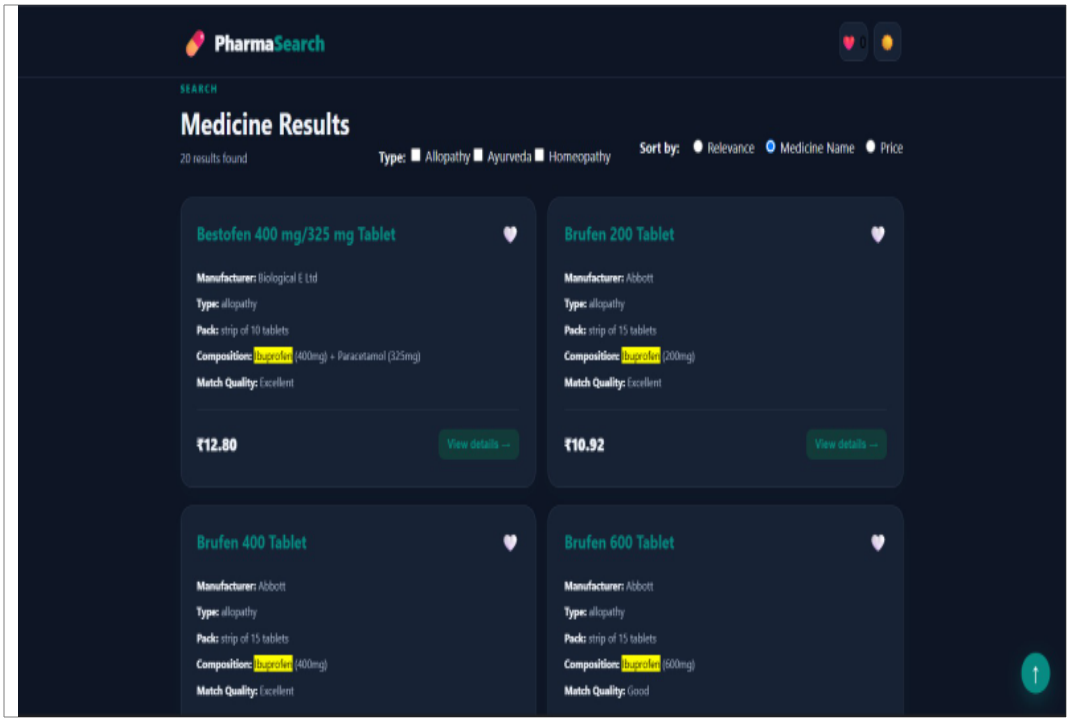


Fig. 5.3 Search Result for an Exact Medicine Name

Analysis: _____

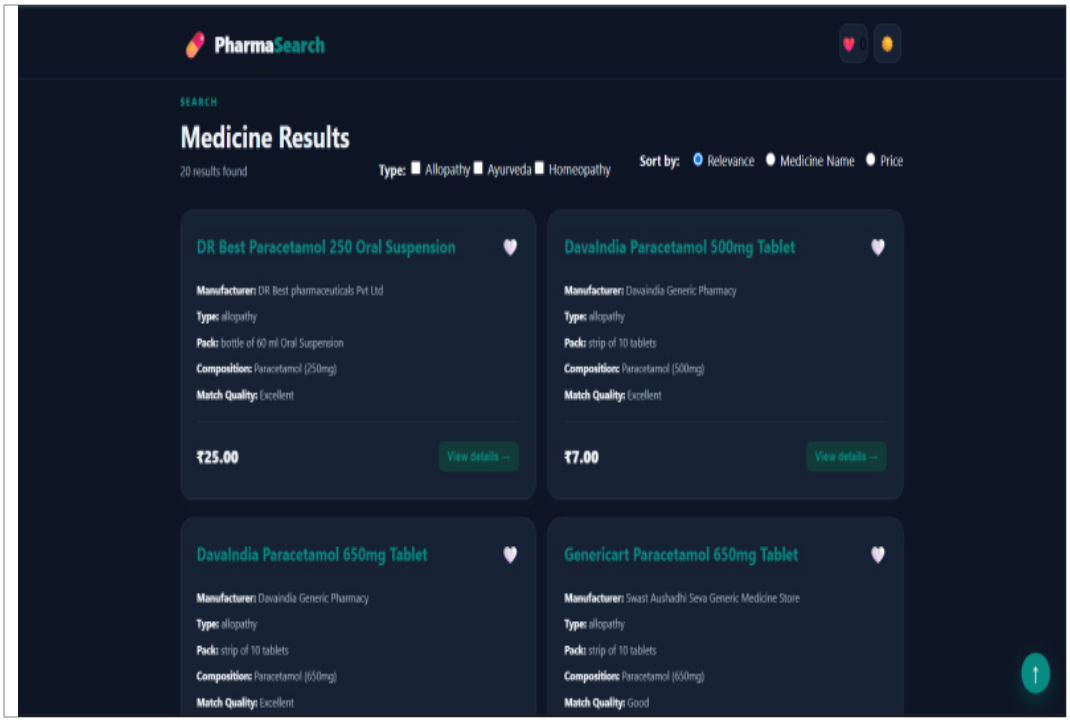


Fig. 5.4 Search Result for a Misspelled Medicine Query

Analysis: _____

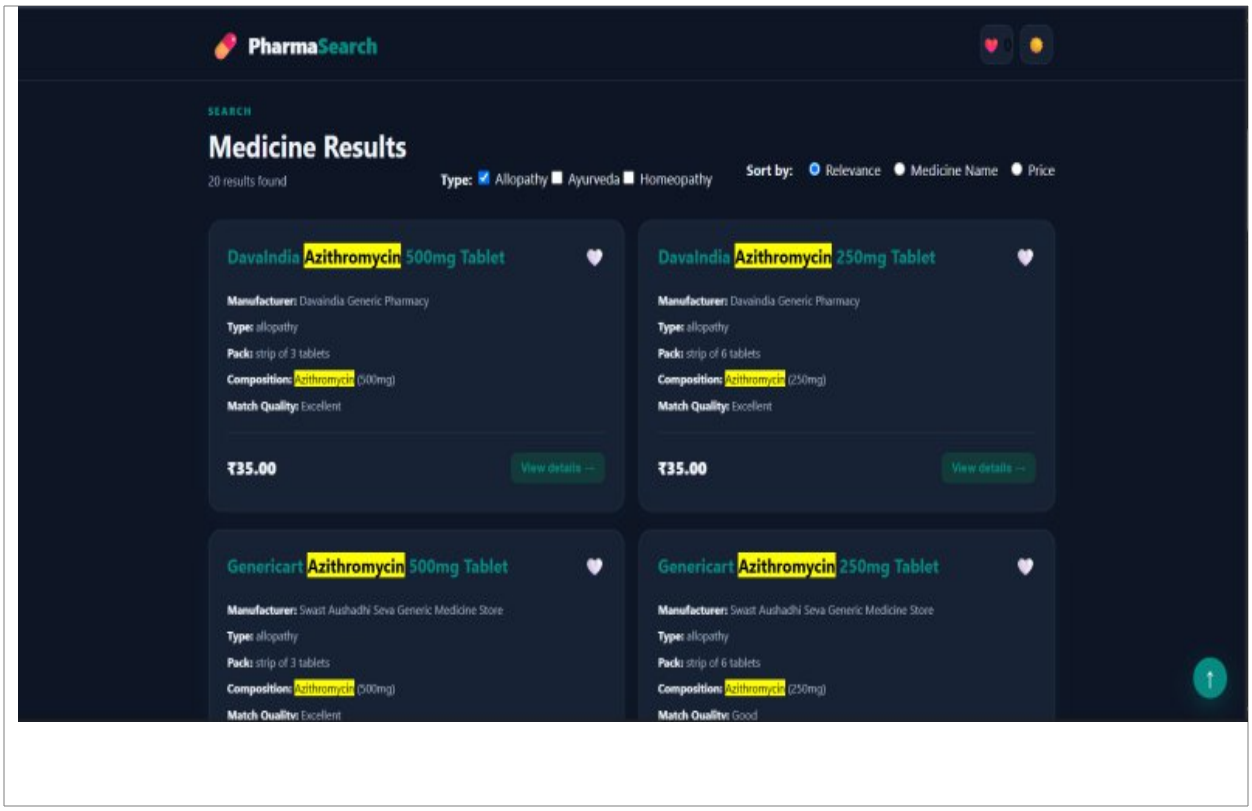


Fig. 5.5 Medicine Type Filtering

Analysis: _____

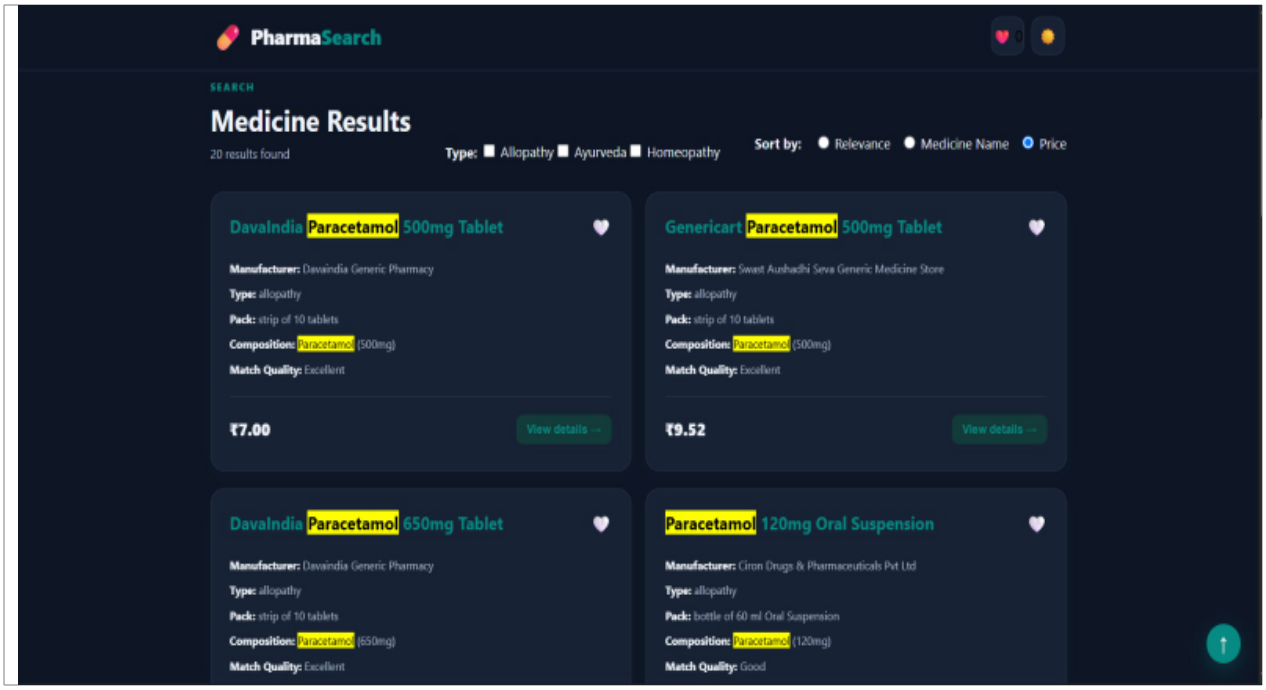


Fig. 5.6 Result Sorting

Analysis: _____

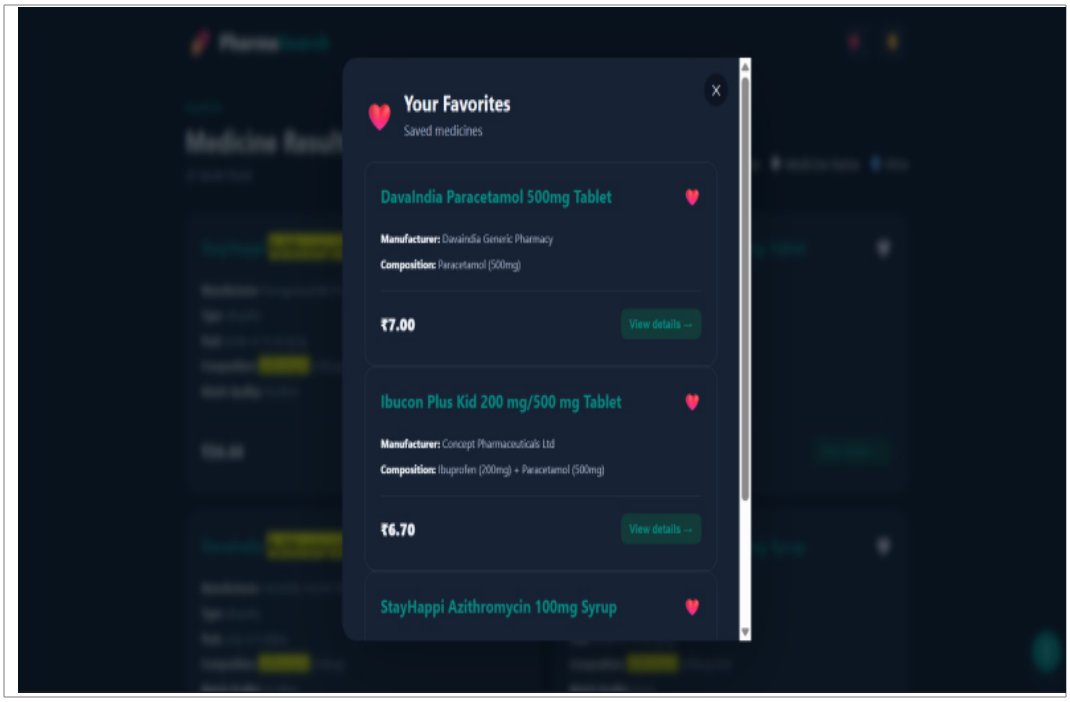


Fig. 5.7 Favourite Medicines

Analysis: _____

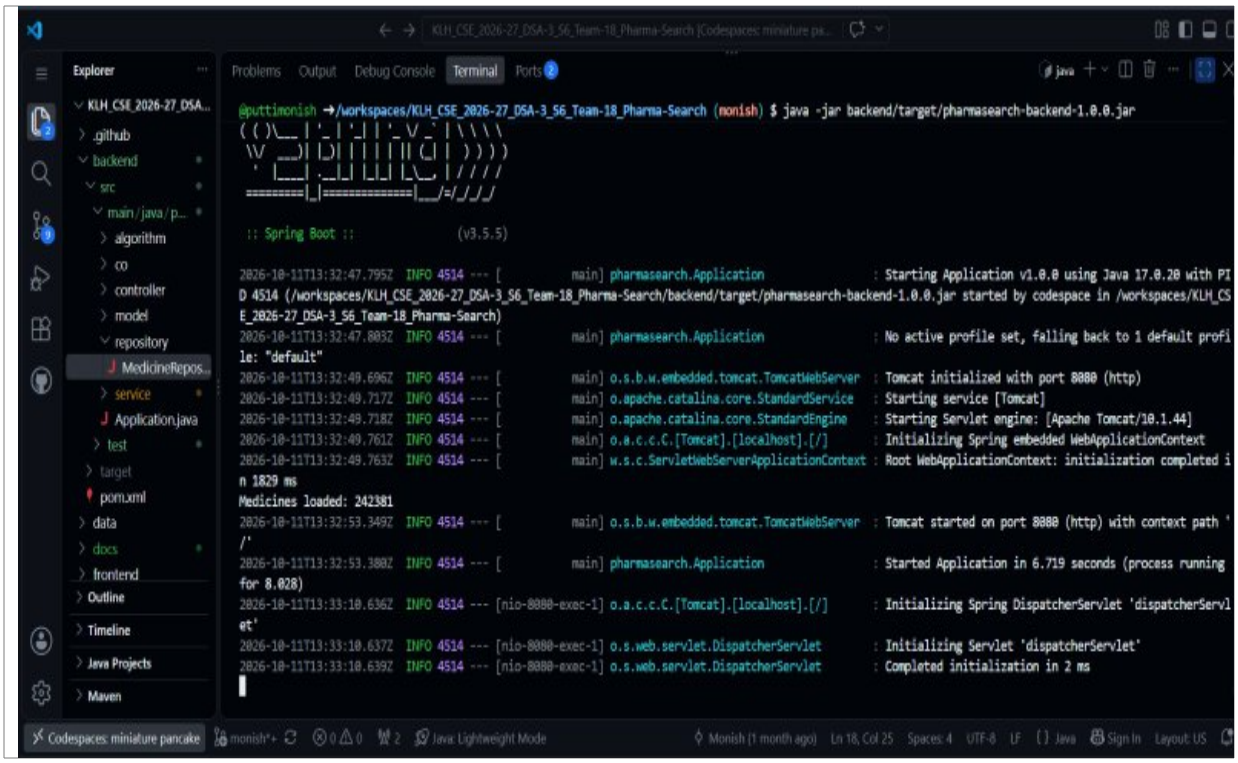


Fig. 5.8 Backend Terminal Showing Medicine Dataset Loading

Analysis: _____

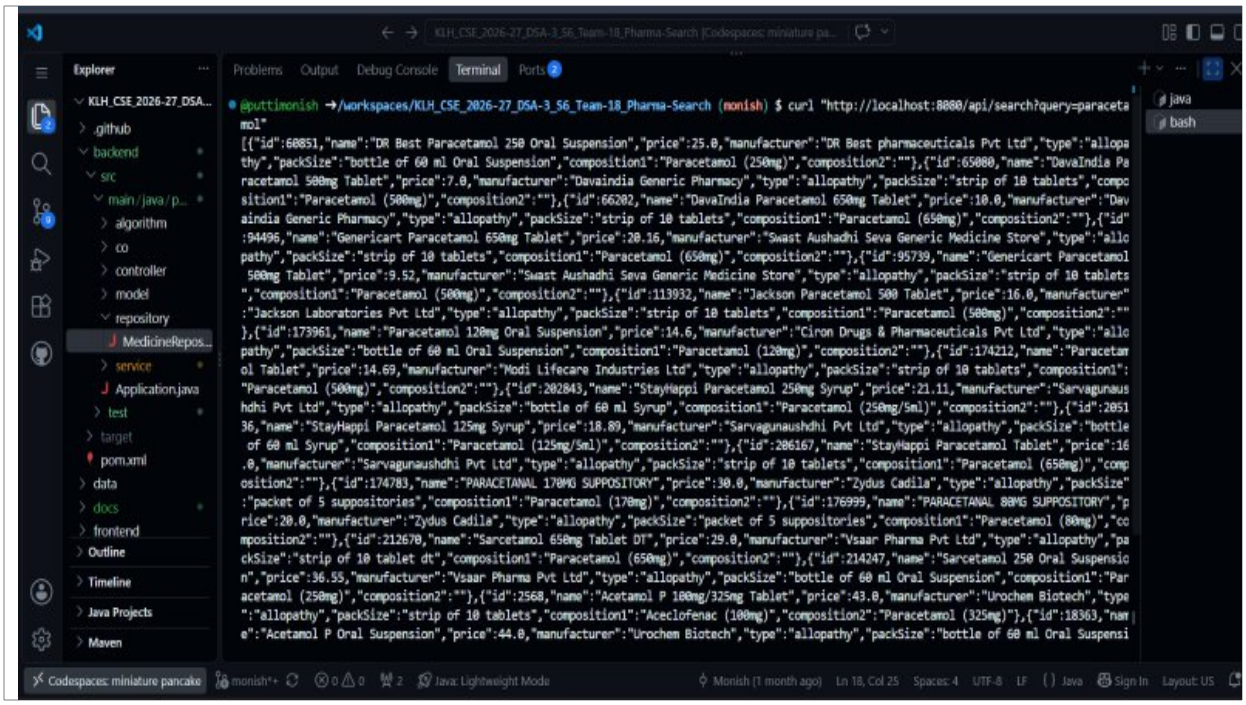


Fig. 5.9 API Search Response (/api/search)

Analysis: _____

6. CHALLENGES AND LIMITATIONS

6.1 Issues Faced During Development

- Processing a large medicine dataset of more than 240,000 records while maintaining acceptable search performance.
- Handling spelling variations, because exact string matching alone cannot identify medicines when the query is misspelt.
- Integrating multiple algorithms without breaking the existing search functionality.
- Frontend-backend communication, because the browser application depends on the Spring Boot backend running on port 8080.

6.2 Solutions Applied

- The dataset was cleaned and stored in a processed CSV file before being loaded by the backend.
- KMP, Rabin-Karp, Edit Distance, and Cosine Similarity were combined to provide different forms of matching.
- A weighted relevance score was introduced to combine the results of multiple matching techniques.
- The frontend communicates with the backend using REST API requests, allowing the interface to display live results.
- Additional algorithm implementations were organised into separate Java classes to keep the project modular and easier to demonstrate.

6.3 Limitations of the System

- The current system uses CSV-based storage rather than a production database.
- The search operation processes the medicine records in memory, which may require optimisation for significantly larger datasets.
- The relevance score is based on predefined weights and may not

always represent human judgement perfectly.

- The Cosine Similarity implementation is based on textual similarity and does not provide complete medical semantic understanding.
- The system is intended primarily as an academic DSA project and is not designed to provide medical diagnosis or treatment recommendations.
- The backend must be running for the browser frontend to retrieve live search results.

7. CONCLUSION AND FUTURE WORK

7.1 Conclusion

Pharma Search demonstrates how Data Structures and Algorithms can be applied to a practical medicine-search problem. The project combines a large medicine dataset, a Java backend, a Spring Boot REST API, a browser-based frontend, and multiple algorithmic techniques into a single application.

KMP, Rabin-Karp, the Z-Algorithm, Edit Distance, and Cosine Similarity demonstrate different approaches to string matching and similarity analysis. The combined ranking mechanism provides more relevant results for both exact and approximate queries. The project also includes implementations corresponding to the broader DSA-3 course outcomes, including network flow, approximation, randomized algorithms, and parallel algorithms.

7.2 Future Work

- Replacing CSV storage with a database for production-scale deployment.
- Improving the relevance-ranking model and the recommendation component.
- Adding more advanced semantic search techniques.
- Improving medicine-name and composition normalisation.

- Adding pagination for very large search-result sets.
- Deploying the Spring Boot backend permanently on a cloud platform.
- Adding detailed medicine information pages and automated performance benchmarking.
- Extending the frontend with additional search and filtering capabilities.

7.3 Skills Learned During Development

- Java programming, Spring Boot development, and REST API development.
- HTML, CSS, and JavaScript frontend development; CSV and file handling.
- KMP string matching, Rabin-Karp rolling hash, the Z-Algorithm, dynamic programming and Edit Distance, and Cosine Similarity.
- Algorithmic result ranking.
- Network-flow, approximation, randomized, and parallel algorithms.
- Git and GitHub version control; debugging and integration testing.
- Connecting algorithmic concepts with a practical software application.

REFERENCES

[1] Oracle, Java Platform, Standard Edition Documentation. [Online]. Available: https://docs.oracle.com/en/java/

[2] J. H. Morris and V. R. Pratt, “A Linear Pattern-Matching Algorithm,” Technical Report 40, University of California, Berkeley, 1970.

[3] D. E. Knuth, J. H. Morris and V. R. Pratt, “Fast Pattern Matching in Strings,” SIAM Journal on Computing, vol. 6, no. 2, pp. 323–350, 1977.

[4] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, MIT Press.

[5] R. A. Wagner and M. J. Fischer, “The String-to-String Correction Problem,” Journal of the ACM, vol. 21, no. 1, pp. 168–173, 1974.

[6] Spring Boot Reference Documentation. [Online]. Available: https://docs.spring.io/spring-boot/

[7] R. M. Karp and M. O. Rabin, “Efficient Randomized Pattern-Matching Algorithms,” IBM Journal of Research and Development, vol. 31, no. 2, pp. 249–260, 1987.

[8] D. Gusfield, Algorithms on Strings, Trees, and Sequences, Cambridge University Press, 1997.

[9] J. Edmonds and R. M. Karp, “Theoretical Improvements in Algorithmic Efficiency for Network Flow Problems,” Journal of the ACM, vol. 19, no. 2, pp. 248–264, 1972.

[10] C. D. Manning, P. Raghavan and H. Schütze, Introduction to Information Retrieval, Cambridge University Press, 2008.

[11] V. I. Levenshtein, “Binary Codes Capable of Correcting Deletions, Insertions, and Reversals,” Soviet Physics Doklady, vol. 10, no. 8, pp. 707–710, 1966.

[12] W. D. Hillis and G. L. Steele Jr., “Data Parallel Algorithms,” Communications of the ACM, vol. 29, no. 12, pp. 1170–1183, 1986.